

AI ASSISTANT CODING ASSIGNMENT-18.2

Name: A SUMANTH

Enrollment no: 2303A52191

Batch: 34

Semester: VI

Branch: CSE(AIML)

Lab 18 – API Integration: Connecting to External Services with Error Handling

Task Description – 1 (Air Quality API Integration)

- **Task:**

Use AI assistance to develop a Python program that retrieves real-time air quality information for a specified city.

Instructions:

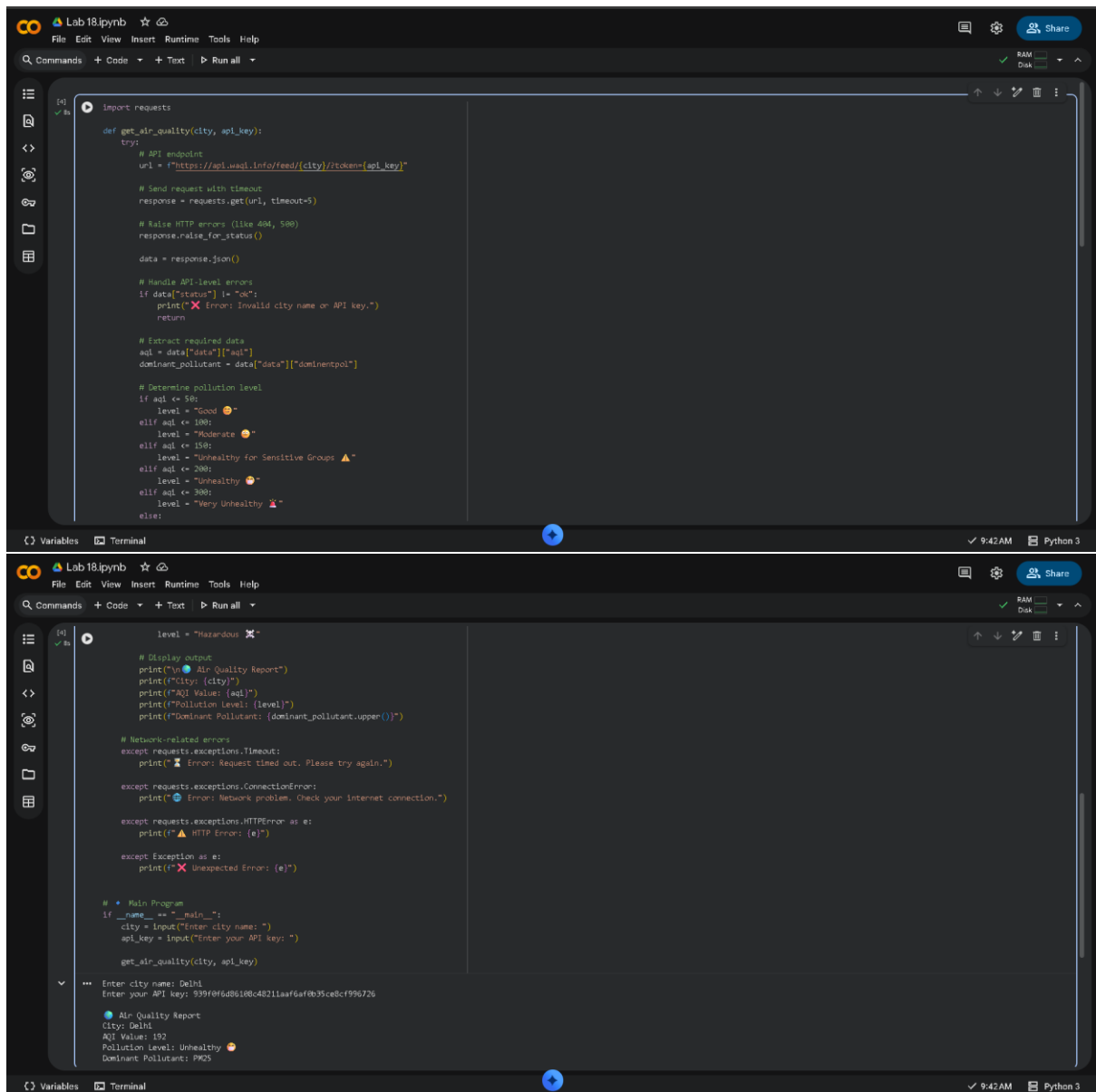
- Generate Python code using the requests library.
- Handle errors such as invalid city names, missing or incorrect API keys, and network timeouts.
- Extract and display AQI value, pollution level, and dominant pollutant in a readable format.

Sample Code:

```
import requests  
  
city = "Delhi"  
  
url = "https://api.example.com/airquality"
```

Expected Output:

- A Python script that successfully fetches and displays air quality data with proper error handling.



Task Description – 2 (Language Translation API)

- Task:

Integrate a Language Translation API to translate text from English to two other languages.

Instructions:

- Use AI to generate API request code for translation.
- Validate user input text and target language codes.
- Handle API response errors and service unavailability.

- Display original and translated text clearly.

Sample Code:

```
import requests
```

```
text = "Welcome to AI Assisted Coding"
```

```
target_lang = "fr"
```

Expected Output:

- A working Python script that translates user input text with graceful error handling.

```
Lab 18(Task 2).ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
RAM Disk
Python 3

import requests

# Supported languages
VALID_LANGUAGES = {
    "fr": "French",
    "es": "Spanish",
    "de": "German",
    "hi": "Hindi",
    "te": "Telugu"
}

def translate_text(text, target_lang):
    try:
        if not text.strip():
            return "❌ Error: Text cannot be empty."

        if target_lang not in VALID_LANGUAGES:
            return "❌ Error: Unsupported language '[target_lang]'. Supported: {list(VALID_LANGUAGES.keys())}"
        url = "https://translate.googleapis.com/translate"

        payload = {
            "q": text,
            "source": "en",
            "target": target_lang,
            "format": "text"
        }

        response = requests.post(url, data=payload, timeout=10)
        if response.status_code != 200:
            return "❌ API Error: Status Code [response.status_code]"

        try:
            data = response.json()
        except:
            return "❌ Invalid API response: [response.text]"
        if "translatedText" not in data:
            return "❌ Error: Unexpected API response format."

        return data["translatedText"]
    except requests.exceptions.Timeout:
        return "❌ Error: Request timed out."
    except requests.exceptions.ConnectionError:
        return "❌ Error: Unable to connect to API."
    except Exception as e:
        return f"❌ Unexpected Error: {str(e)}"

# MAIN PROGRAM
if __name__ == "__main__":
    print("🔹 Language Translation Tool")
    print("Supported languages:", VALID_LANGUAGES)

    text = input("Enter text in English: ").strip()

    lang1 = input("Enter first target language code: ").strip().lower()
    lang2 = input("Enter second target language code: ").strip().lower()

    print("\n • Original Text:", text)

    result1 = translate_text(text, lang1)
    result2 = translate_text(text, lang2)

    print("\n • Translated to (VALID_LANGUAGES.get(lang1, lang1)): (result1)")
    print("\n • Translated to (VALID_LANGUAGES.get(lang2, lang2)): (result2)")

    print("\n 🔹 Language Translation Tool")
    print("Supported Languages: {'fr': 'French', 'es': 'Spanish', 'de': 'German', 'hi': 'Hindi', 'te': 'Telugu'}")
    print("Enter text in English: Welcome to AI Assisted Coding")
    print("Enter first target language code: fr")
    print("Enter second target language code: hi")

    print("\n • Original Text: Welcome to AI Assisted Coding")
    print("\n • Translated to French: ❌ Error: Unable to connect to API.")
    print("\n • Translated to Hindi: ❌ Error: Unable to connect to API.")
```

Task Description – 3 (Public Book Information API)

- Task:

Connect to a public Books API to fetch information about a book using its title or ISBN.

Instructions:

- Prompt AI to write Python code for sending GET requests to the API.
- Handle errors such as book not found, invalid input, and rate limits.
- Display book title, author, publisher, and publication year.

Sample Code:

```
import requests
```

```
query = "Artificial Intelligence"
```

Expected Output:

- A Python program that retrieves and displays book details with robust error handling.

```
BASE_URL_TITLE = "https://openlibrary.org/search.json"
ISBN_URL = "https://openlibrary.org/api/books?bibkeys=ISBN:%s&format=json&cmd=detail"

print("Open Library API Base URLs defined.")
Open Library API Base URLs defined.

Now, let's define a function to search for a book by its title. It'll include comprehensive error handling for network issues and cases where no book is found.

import requests

def search_book_by_title(book_title):
    print(f"Searching for book by title: '{book_title}'...")
    params = {'title': book_title, 'limit': 1} # limit to 1 result for simplicity
    try:
        response = requests.get(BASE_URL_TITLE, params=params)
        response.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
        data = response.json()

        if data['numfound'] > 0 and 'docs' in data and len(data['docs']) > 0:
            book = data['docs'][0]
            print(f"Book Details (by Title Search):")
            print(f"Title: {book.get('title', 'N/A')}")
            # Authors can be a list, join them if multiple
            authors = book.get('author_name', ['N/A'])
            print(f"Author(s): {' '.join(authors)}")
            # Publisher is often a list, take the first one
            publisher = book.get('publisher', ['N/A'])
            print(f"Publisher: {publisher[0] if publisher else 'N/A'}")
            print(f"Publication Year: {book.get('first_publish_year', 'N/A')}")
        else:
            print(f"Error: No book found with title '{book_title}'")

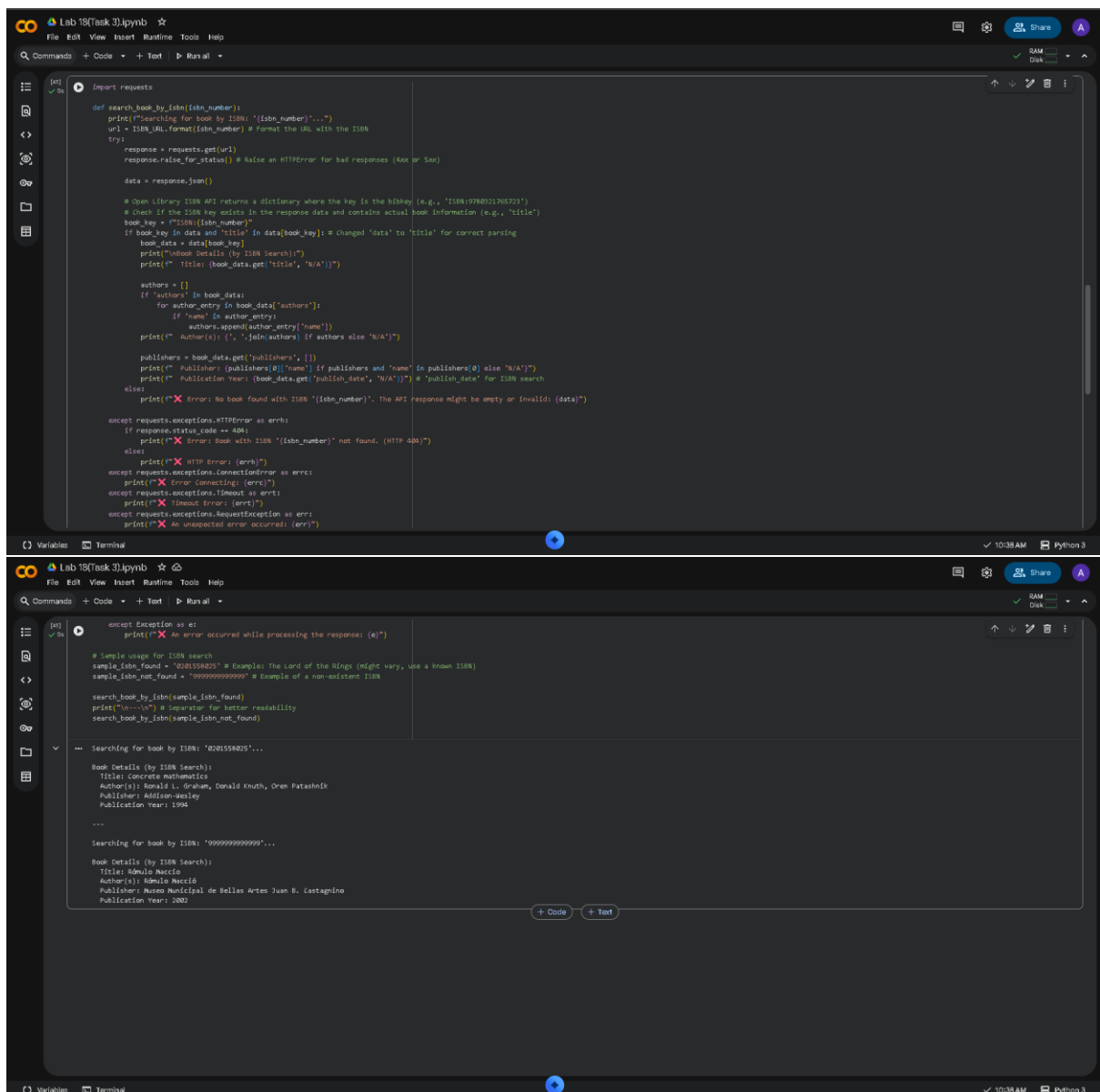
    except requests.exceptions.HTTPError as errh:
        print(f"HTTP Error: {errh}")
    except requests.exceptions.ConnectionError as errc:
        print(f"Error connecting: {errc}")
    except requests.exceptions.Timeout as errt:
        print(f"Timeout Error: {errt}")
    except Exception as err:
        print(f"An error occurred while processing the response: {err}")

# Sample usage for title search
query_title = "Artificial Intelligence"
search_book_by_title(query_title)

query_title_not_found = "A Very Oscure Book That Does Not Exist 12345"
search_book_by_title(query_title_not_found)

Searching for book by title: 'Artificial Intelligence'...

Book Details (by Title Search):
Title: Artificial Intelligence
Author(s): Stuart J. Russell, Peter Norvig
Publisher: N/A
Publication Year: 1994
```



Task Description – 4 (Real-Time Application: Job Listings Aggregator)

• Scenario:

Develop a Job Listings Aggregator using a public Jobs API.

Requirements:

- Fetch the latest job postings for a given role or location.
- Handle errors such as invalid search terms, missing API keys, and request failures.
- Display job title, company name, location, and posting date in a numbered list.

- Implement a retry mechanism for failed API requests.

Sample Code:

```
import requests
```

```
keyword = "Python Developer"
```

Expected Output:

- A functional Python script that retrieves and displays job listings with proper error handling.

The image shows two screenshots of a JupyterLab notebook interface. The top screenshot displays the initial code setup, including the installation of the 'requests' library and the definition of a 'fetch_jobs' function with a retry mechanism. The bottom screenshot shows the execution of the script, which prompts the user to enter a job role and displays the resulting job listings.

```
[50] pip install requests
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4, >=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.6)
Requirement already satisfied: idna<4, >=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3, >=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi<=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2026.2.25)

[70] import requests
import time

BASE_URL = "https://www.arbeltnew.com/api/job-board-api"

def fetch_jobs(keyword, retries=3):
    keyword = keyword.lower()
    for attempt in range(retries):
        try:
            response = requests.get(BASE_URL, timeout=5)
            if response.status_code != 200:
                raise Exception("API request failed")
            data = response.json()
            jobs = []
            for job in data.get("data", []):
                title = job.get("title", "").lower()
                if any(word in title for word in keyword.split()):
                    jobs.append(job)
            return jobs[:10]
        except requests.exceptions.Timeout:
            print(f"Retry (attempt={attempt+1}): Timeout...")
        except requests.exceptions.ConnectionError:
            print(f"Retry (attempt={attempt+1}): Connection error...")

    except Exception as e:
        print(f"Error: {e}")
        break
    time.sleep(2)
    return []

def display_jobs(jobs):
    if not jobs:
        print("❌ No jobs found. Try different keyword like 'developer', 'engineer'")
        return
    print("\n🌟 Job Listings:\n")
    for i, job in enumerate(jobs, 1):
        print(f"{i}. {job.get('title', 'N/A')}")
        print(f"   🏢 Company: {job.get('company_name', 'N/A')}")
        print(f"   📍 Location: {job.get('location', 'N/A')}")
        print(f"   📅 Posted: {job.get('created_at', 'N/A')}")
        print(f"   🕒 {job.get('created_at', 'N/A')}")

# MAIN
keyword = input("Enter job role: ").strip()
if keyword:
    jobs = fetch_jobs(keyword)
    display_jobs(jobs)
else:
    print("❌ Invalid Input")

Enter job role: Python Developer

🌟 Job Listings:
1. Software Developer In Test II - Java (m/f/d)
   🏢 Company: Onapsis
   📍 Location: Heidelberg, Baden-Württemberg, Germany
   📅 Posted: 1774878315
-----
2. Ruby on Rails Developer (m/w/d)
   🏢 Company: alfinyo AG
   📍 Location: Dresden
   📅 Posted: 1774875689
-----
```

Task Description – 5 Stock Market Data API Integration

- Task:

Use AI assistance to develop a Python program that retrieves real-time stock market data for a selected company.

Instructions:

- Generate Python code using the requests library to connect to a public Stock Market API.
- Validate user input for stock symbol.
- Handle common errors such as invalid symbols, API rate limits, missing API keys, and network failures.
- Extract and display stock name, current price, day high, day low, and percentage change.

Sample Code:

```
import requests  
  
symbol = "AAPL"
```

- Expected Output:

A Python script that successfully fetches and displays stock market information with robust error handling.

```
Lab 18(Task 5).ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text + Run all
RAM Disk

[In] 1
import requests
import time

API_KEY = "YOUR_API_KEY_HERE"
ALPHA_URL = "https://www.alphavantage.co/query"

def validate_symbol(symbol):
    return symbol.isalnum()

def fetch_from_alpha(symbol):
    if API_KEY == "YOUR_API_KEY_HERE" or not API_KEY:
        return None
    params = {
        "function": "GLOBAL_QUOTE",
        "symbol": symbol,
        "apikey": API_KEY
    }
    try:
        response = requests.get(ALPHA_URL, params=params, timeout=10)
        data = response.json()
        if "Note" in data:
            print("▲ Alpha Vantage rate limit reached.")
            return None
        if "Error Message" in data:
            print("✗ Invalid Alpha API key.")
            return None
        quote = data.get("Global Quote")
        if not quote:
            return None
        return {
            "source": "Alpha Vantage",
            "symbol": quote.get("01. symbol"),
            "price": quote.get("05. price"),
            "high": quote.get("03. high"),
            "low": quote.get("04. low"),
            "change_percent": quote.get("10. change percent")
        }
    except Exception as e:
        print("✗ Alpha error:", e)
        return None

def fetch_from_fmp(symbol):
    url = "https://financialmodelingprep.com/api/v3/quote/{symbol}?apikey=demo"
    try:
```

```
Lab 18(Task 5).ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text + Run all
RAM Disk

[In] 2
try:
    response = requests.get(url, timeout=10)
    if response.status_code != 200:
        print("✗ API request failed.")
        return None
    data = response.json()
    if not data:
        print("✗ Invalid stock symbol.")
        return None
    stock = data[0]
    return {
        "source": "FMP API",
        "symbol": stock.get("symbol"),
        "price": stock.get("price"),
        "high": stock.get("high"),
        "low": stock.get("low"),
        "change_percent": str(round(stock.get("changesPercentage", 0), 2)) + "%"
    }
except requests.exceptions.Timeout:
    print("⌚ Request timed out.")
except requests.exceptions.ConnectionError:
    print("🌐 Network error.")
except Exception as e:
    print("✗ FMP error:", e)
    return None

def display_stock(data):
    print("\n📊 Stock Market Data")
    print("-----")
    print(f"Source: {data['source']}")
    print(f"Symbol: {data['symbol']}")
    print(f"Current Price: {data['price']}")
    print(f"Day High: {data['high']}")
    print(f"Day Low: {data['low']}")
    print(f"Change: {data['change_percent']}")

def main():
    symbol = input("Enter stock symbol (e.g., AAPL, TSLA): ").strip().upper()
    if not validate_symbol(symbol):
        print("✗ Invalid symbol. Use only letters/numbers.")
```

```
Lab 18(Task 5).ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text + Run all
RAM Disk

[In] 3
return
retries = 3
for attempt in range(retries):
    print(f"⏱️ Attempt {attempt + 1}...")
    # Try Alpha (optional)
    data = fetch_from_alpha(symbol)
    # fallback to FMP (main working API)
    if not data:
        print("🔄 Switching to FMP API...")
        data = fetch_from_fmp(symbol)
    if data:
        display_stock(data)
        return
    else:
        if attempt < retries - 1:
            print("🔄 Retrying in 2 seconds...")
            time.sleep(2)
        print("❌ All retry attempts failed.")
        print("⚠️ Possible issues:")
        print("  - No Internet connection")
        print("  - API temporarily down")
        print("  - Invalid stock symbol")
    if __name__ == "__main__":
        main()

Enter stock symbol (e.g., AAPL, TSLA): AAPL
Attempt 1...
Switching to FMP API...
API request failed.
Retrying in 2 seconds...
Attempt 2...
Switching to FMP API...
API request failed.
Retrying in 2 seconds...
Attempt 3...
Switching to FMP API...
API request failed.
All retry attempts failed.
Possible issues:
- No Internet connection
- API temporarily down
- Invalid stock symbol
```